values in the state array are exactly 0. This might also help with *overfitting*, which is when a model learns by memorizing spurious details in the data without learning the abstract features of the data, ultimately preventing it from generalizing to new data.

There are a couple of things to point out that you may not have seen before. The first new thing is the use of the context `torch.no_grad()` when computing the next state Q value. Whenever we run a PyTorch model with some input, it will implicitly create a computational graph. Each PyTorch tensor is not only a store of tensor data, it also keeps track of which computations were performed to produce it. By using the `torch.no_grad()` context, we tell PyTorch to *not* create a computational graph for the code within the context; this will save memory when we don't need the computational graph. When we compute the Q values for `state2`, we're just using them as a target for training. We're not going to backpropagate through the computational graph that would have been created if we hadn't used `torch.no_grad`. We only want to backpropagate through the computational graph that is created when we call `model(state1),` because we want to train the parameters with respect to `state1`, not `state2`.

Here's a simple example with a linear model:

```
>>> m = torch.Tensor([2.0])
>>> m.requires_grad=True
>>> b = torch.Tensor([1.0])
>>> b.requires_grad=True
>>> def linear_model(x,m,b):
>>>    y = m @ x + b
>>>    return y
>>> y = linear_model(torch.Tensor([4.]), m,b)
>>> y
tensor([9.], grad_fn=<AddBackward0>)
>>> y.grad_fn
<AddBackward0 at 0x128dfb828>
>>> with torch.no_grad():
>>>    y = linear_model(torch.Tensor([4]),m,b)
>>> y
tensor([9.])
```